Develop a Web application to manage information related to the collection, sorting, and recycling of waste – by category: household, paper, plastic, etc. – at the level of citizens, authorized personnel, and decision-makers. The system will support user reporting of locations where a substantial amount of garbage has accumulated, with the goal of facilitating clean-up efforts. At set time intervals (day, week, month), numerical and graphical reports will be generated – available in HTML, CSV, and PDF formats – regarding the current situation at the neighborhood/locality level, highlighting the cleanest and dirtiest areas.

**Documentation:**

1. **Introduction**
   **1.1. Goal**

The goal of the project is developing a web application to support the cleanliness of the cities, while people can contribute to the community. The application gives the citizens the ability to report information about locations where there is significant waste, while the authorized personnel will monitor and review the reports and mark them as solved when the waste area has been cleared by the authorities.
The application offers a common communication channel between the citizens and the authorities: quick way to signal dirty areas and quick response from the city's personnel, increasing overall efficiency of managing the waste and recycling process.

**1.2. Problem**

Urban areas have significant waste accumulation, posing a challenge to authorities and citizens. Although there are specific recycling systems and waste collection bins available in the cities and in frequent locations, the waste related issues are still persistent. These include illegal dumping, uncollected garbage,improper sorting and incorrect use of the containers.

Worldwide, the concept of recycling appeared in the late 19th century and massively used after the resource crisis in 1970. While some countries have strict laws and are determined to recycle correctly, other countries don't apply the laws. For example, Romania recorded a recycling rate of 12%, while other European Union countries average around 50%. This is a reflection on the challenges related to public participation, authority efficiency and education on sorting practices.

The absence of recycling systems is not the problem, but the awareness and participation of the citizens. While some demographic categories don't give much thought about this, a wide range of people would conform to the laws, if the system allowed them to communicate more clearly about the dirty areas in their cities, if the authorities listened to them and solved the problems promptly. Feedback for people is as important as the cause. The web application, with its interactive features could motivate in this direction.

The new system allows reporting, centralizes information in a database, allows the user to pin-point the wasteful areas on map, provides visual reports and tracking status and statistical analysis.

### 1.3. Users

The web application is designed for three types of users, each of them having different roles, permissions and responsibilities in the system.

The first category of users is Citizen. They are common users of the system. These users can create an account, authenticate, report waste accumulation in specific locations, they can view reports on the map and track the status of their reports. They can see on the map in the user interface, when a report has been solved.

The second category of users is Authorized Personnel. They are responsible for resolving reported issues. They can access and review reports, monitor their status, and update the status.

The third user category is the Administrator. They have full access to the system and are responsible for handling users, managing all reports and activities, generating statistics and analyzing them.

## 2. Functional Requirements Overview

The web application will have the following functionalities:

- User registration and authentication
- Allows users to report waste locations and details such as the category, description
- Visualisation of reports, in list format and on a map
- Allows authorized personnel to update the status of reports
- Allows administrators to manage users and generate analytical reports
- Provides statistical analysis and data visualisation based on reports
- Supports filtering and searching in the report list
- Allows exporting data in formats as CSV and PDF

## 3. Non-Functional Requirements Overview

The following requirements ensure that the application is reliable, efficient, secure and user-friendly.

**3.1. Performance:** The system should be fast and have efficient response times for all user interactions. API requests, creating, retrieving, updating and deleting reports should be processed fast and ensure a smooth user experience.

**3.2. Security:** The system should ensure the safety of user data and prevent unauthorized events. Authentication and authorizations are implemented to restrict access based on user roles. Sensitive data such as passwords are securely stored using password hashing. Protection against web vulnerabilities that need to be implemented include: SQL injection (using prepared statements), Cross-Site Scripting (XSS) (using output sanitization) and invalid or malicious input (through validation).

**3.3. Usability:** The system needs to be easy to use, intuitive and user-friendly, allowing the users to navigate easily within the application. The interface needs to be consistent, simple, and actions such as reporting waste or viewing reports should have to be handled without any

confusion. Error messages and feedback have to be clear, informative. The system overall must be accessible to users regardless of their background, demographics or technical experience.

**3.4. Scalability:** The system should be designed to support future evolution. It should handle a large number of users, reports and data, without crashing or needing to change the whole architecture. The project structure should allow expansion for new features. The database should support efficient querying and a large amount of users and data.

**3.5. Responsiveness:** The system needs to have a responsive design, to ensure that all functionalities can be used on different devices or screen sizes, such as desktop, tablets, mobile devices, without losing any functionalities or usabilities. The layout should be dynamic and interactive elements should be accessible and easy to use.

## 4. Use cases

The system supports three main categories of users: citizens (regular users), authorized personnel and administrators. Each category has different permissions and responsibilities within the application.

**4.1. Citizens, regular users:** interact with the system mainly for reporting waste-related issues and monitoring existing reports. A regular user has numerous abilities and permissions in the system.

**Register:** The user is able to create an account. They must provide required data: username, email address and a password. The system validates the input, stores the data securely in the database and can retrieve it at authentication.
**Login:** The user is able to log in using the credentials stored after the registering process is finished. When a user logs in, the system creates a session and gives them access to the user specific functionalities.
**Report Waste**: The user can report and submit a waste related issue. The report is a form that requires details such as the location (address or pinpoint on the map), waste category and a short description of the issue. After submitting a report, the system stores it in the database and is given the pending status, until the issue is resolved by an authorized worker.
**View Reports:** The user is able to view all reports in the system, which are shown on the map, using pins, and in a list format. They can interact with the reports to view details: description, category and status.
**Track report status:** The user can monitor the status of their submitted reports and be notified when a report is marked as solved.

**4.2. Authorized Personnel:** they are responsible for handling and resolving reported issues by the regular users, and have specific permissions and abilities within the application.

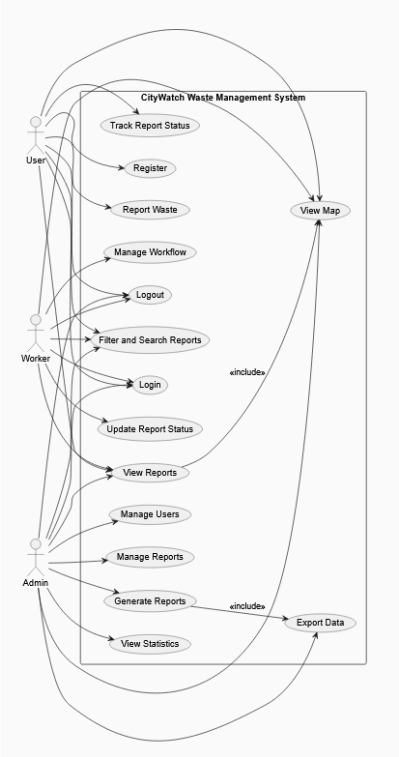
**Login:** The worker is able to log in using the credentials provided by the administrator, and is given access to worker-specific functionalities.
**View Reports**: The worker can access and review all reports submitted by the users.

**Update Report Status:** The worker can modify and update the status of a report: pending, in progress and resolved. When a report is solved, the system updates the database and changes the status in the user interface as well.
**Manage Workflow:** The authorized personnel can prioritize reports based on urgency or location for an efficient workflow and clean community.

**4.3. Administrators:** This type of user has full access and control over the system and manages and does the analysis.

**Manage Users:** Administrators can create new users, update existing users, delete users, assign roles.
**View Reports:** They have access to all reports and can monitor activity within the application.
**Manage Reports:** Administrators can edit and delete reports if necessary: if they duplicate or if the submission was invalid.
**Generate Reports**: Administrators are able to generate analytical reports based on the collected data.
**View Statistics:** The administrators are able to visualize statistical data: number of reports, categories, clean versus dirty areas.
**Export Data:** They can export data in formats: CSV or PDF.

**4.4. UML Diagrams**




***Interactions between the system and the actors***

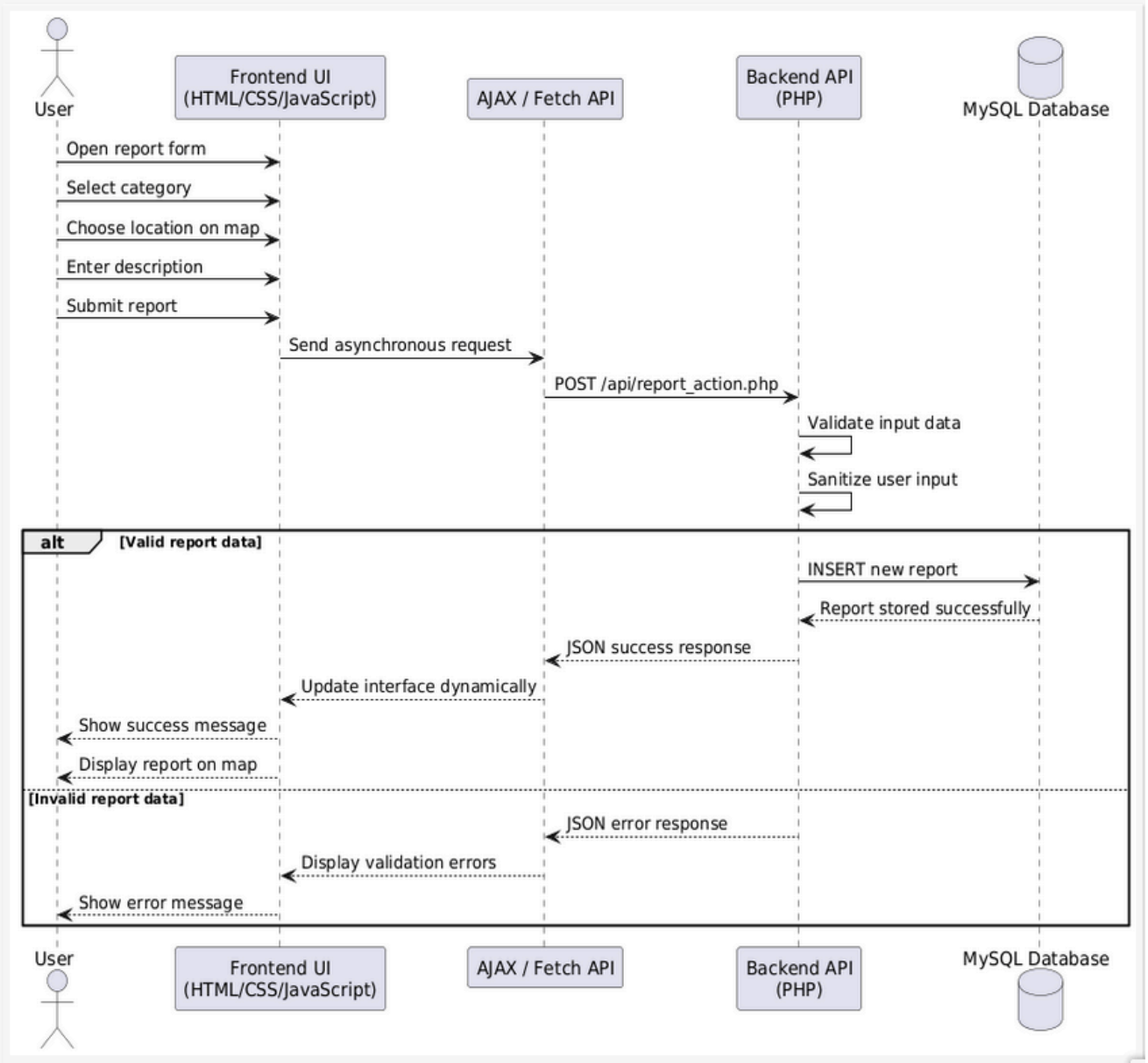



***Diagram that shows the process of reporting a waste-related issue***

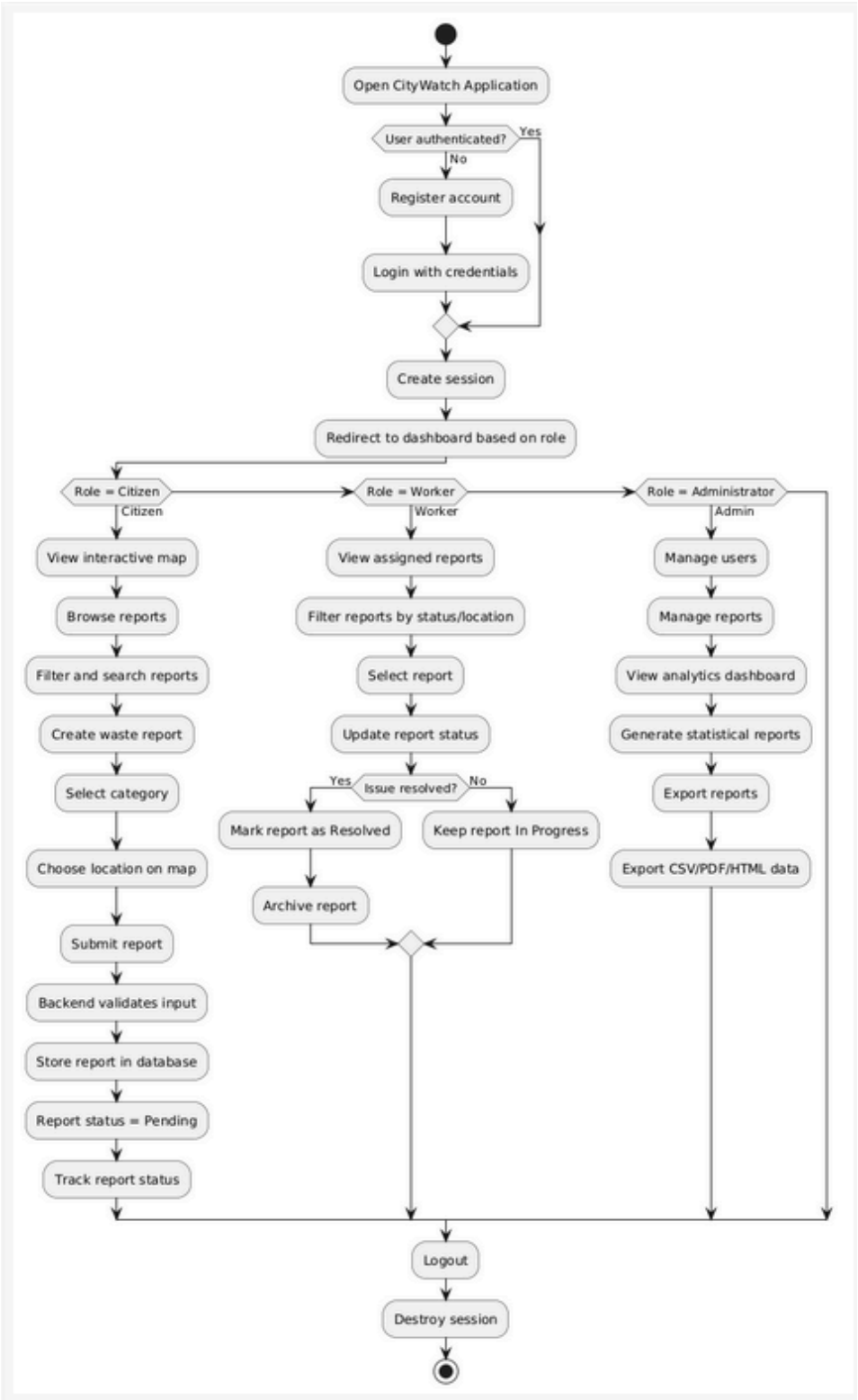



***Diagram shows the workflow of the application***

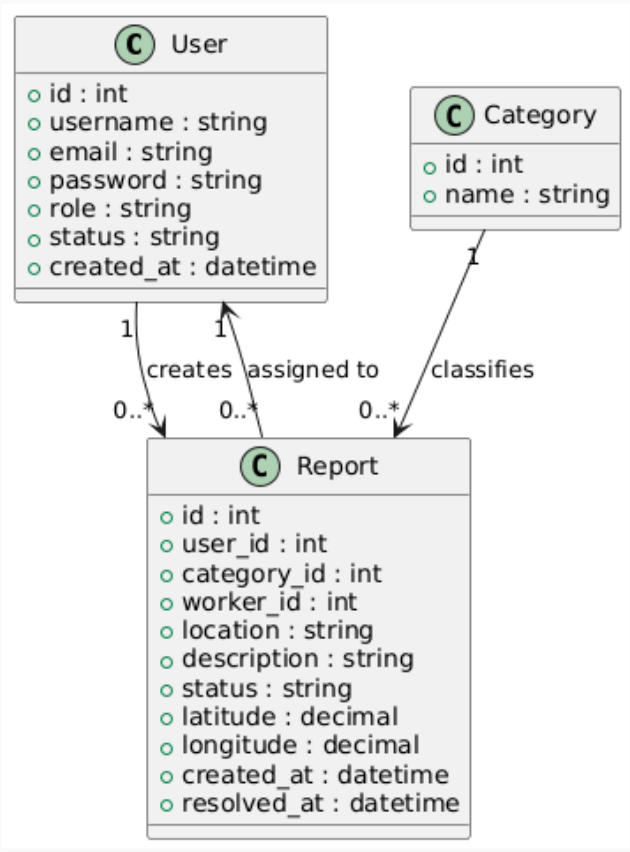



***Diagram that shows the main structures of the system and their relationships***

## 5. System Architecture

The application is based on the client-server architecture model: the client, the frontend, communicates and exchanges information securely over the internet, with the server, the backend, through HTTP API endpoints. The frontend is responsible for handling user interactions and displaying data and the backend is responsible for business logic, data processing and communication with the database. The two communicate using HTTP requests.

**5.1. Frontend (HTML, CSS, JavaScript)** is the client-side of the web application. It renders the user interface, handles user interactions and sends requests to the server. The interface uses HTML for website structure, CSS for styling and JavaScript for dynamic elements and communication.
**5.2. Backend (PHP API)** is implemented using PHP and is responsible for processing requests, validating input data, business logic and database interaction.
**5.3. Database (MySQL)** is responsible for storing all data: users, reports, categories.
**5.4. Data Flow:** When the user performs an action, the frontend accepts the input data and sends a HTTP request to the backend. The backend receives the request, validates the data, processes it. It interacts with the database and stores or retrieves the information. After this, the backend sends a response to the frontend in a JSON format. The frontend updates the interface and displays the action result.

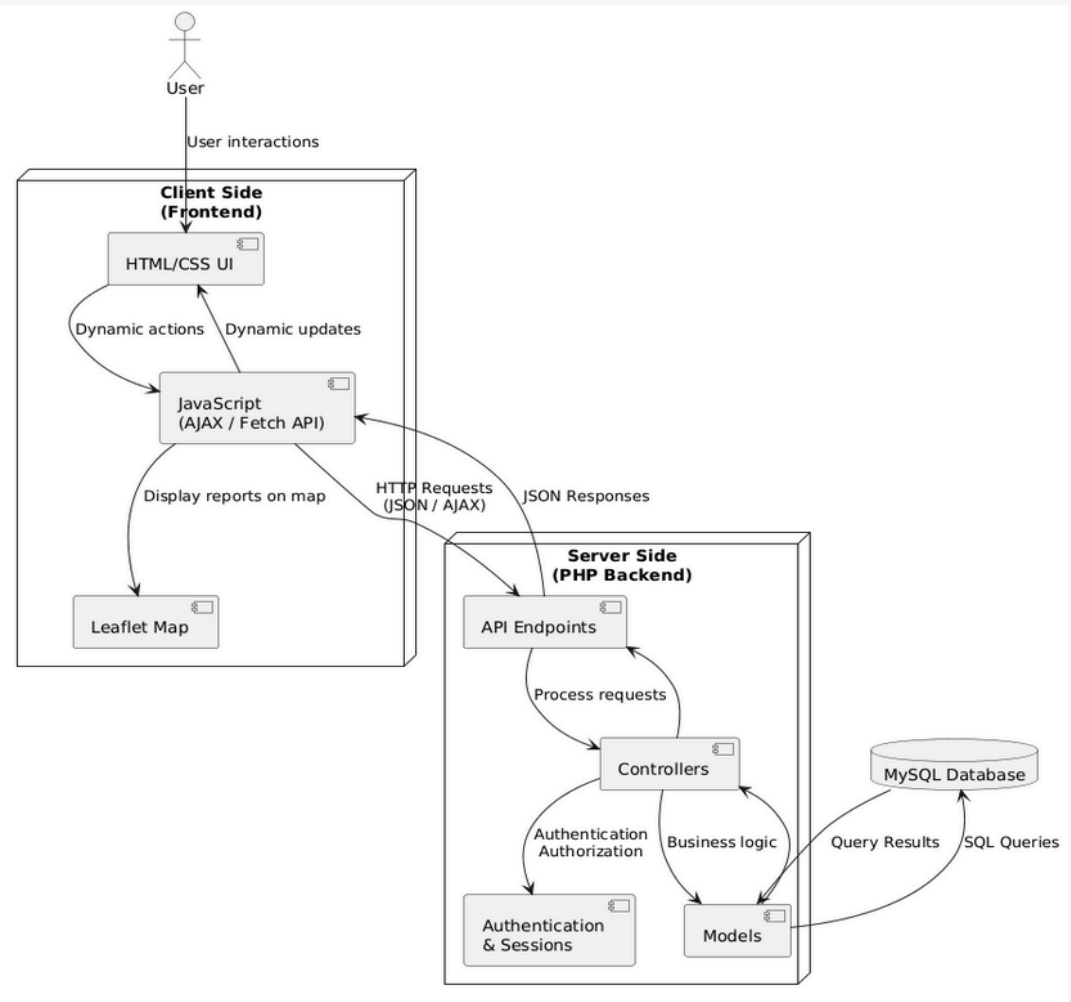



***Diagram that shows the interaction between the three main components***

### 6. Database

The database is responsible for storing and managing all the data. It is implemented using MySQL. Data is organized in tables: users, reports and categories.

**6.1. Users Table**: stores information on all users: citizens, authorized personnel and administrators. Attributes:

- *id (integer, primary key)*: unique identifier for each user
- *username (string)*: the name used for authentication
- email (string): the email used for authentication
- *password (string)*: hashed password
- *role (string)*: type of user
- *status (string)*: active or inactive

**6.2. Reports Table:** stored all waste-related reports submitted by the users. Attributes:

- *id (integer, primary key)*: unique identifier for each report
- *user_id (integer, foreign key)*: reference to the user who created the report
- *category_id (integer, foreign key)*: references the type of waste report

- *worker_id (integer, foreign key)*: references the worker assigned to the report
- *location (string)*: text description, address of the location of the waste
- *description (string)*: details about the issue
- *status (string)*: current status of the report: pending, progress, resolved
- *created_at (datetime)*: a timestamp of the creation of the report

**6.3. Categories Table:** used to define the different types of waste that can be reported. Attributes:

- *id (integer, primary key):* unique identifier for each category
- *name (string):* name of the category: plastic, paper, glass, household

**6.4. Relationships:**

- A user can create multiple reports. A report belongs to one user: relationship one-to-many (1:N) between Users and Reports tables.
- A report belongs to a category, a category can be linked to multiple reports: relationship one-to-many (1:N) between Categories and Reports tables.
- A worker can handle multiple reports (1:N).

## 7. API endpoints

The backend of the application has multiple API endpoints used for communication between the frontend and the server. These endpoints process HTTP requests, interact with the database and return responses in JSON format. The API architecture allows asynchronous communication using AJAX requests, improves responsiveness and user experience by avoiding full page reloads.

This application mainly uses GET and POST requests as HTTP methods.

- GET requests: are used for retrieving data from the server. These include retrieving reports, filtering reports based on category, status, loading users and displaying statistical information.
- POST requests: are used for creating new resources and processing actions such as user authentication, report submission, updating report statuses and admin operations. Form data is sent asynchronously from the frontend to the backend API.

Examples of API endpoints:

- GET /api/get_reports.php: retrieves all reports from the database
- GET /api/filter_reports.php: filters reports dynamically using query parameters
- POST /api/report_action.php: updates report statuses
- POST /api/user_action.php: performs admin user operations
- POST /pages/login.php: authenticates users and creates sessions.

All endpoints validate user input, use prepared SQL statements for database security and return structured responses used dynamically by the frontend interface.

## 8. Backend

The backend of the application is implemented using PHP and is organized into separate files for API endpoints, controllers, models and configuration utilities. It is responsible for processing requests, validating input data, implementing business logic, handling authentication and

interacting with the MySQL database. Communication between frontend and backend is performed asynchronously using AJAX requests and JSON responses.

**Components**:

- **Routes**: the backend shows multiple API endpoints that act as routes for frontend requests. These endpoints receive HTTP requests (GET and POST), process the required actions and return responses in JSON format. (e.g. /api/get_reports.php, /api/filter_reports.php, /pages/login.php).
- **Controllers**: contain the main application logic and process user actions received from the frontend. They validate input data, handle authentication and coordinate communication between models and views/pages. (e.g. AuthController.php, ReportController.php, UserController.php).
- **Models**: are responsible for database interaction and data management. They contain SQL queries, prepared statements, functions used for retrieving, inserting, updating and deleting data from the database. (e.g. UserModel.php, ReportModel.php, CategoryModel.php).
- **Utility and configuration files:** provide functionality used throughout the application. (e.g. db.php for database connection, session.php for session and cookie handling, and helper functions for validation and security.

**Data Flow**:

The frontend sends asynchronous HTTP requests to backend API endpoints using JavaScript fetch/AJAX calls. The backend receives the request, validates and sanitizes the data and processes the required business logic through controllers and models. The models interact with the MySQL database using prepared SQL statements. After processing, the backend returns a JSON response to the frontend, which dynamically updates the user interface without reloading the page.

**Response:**

The backend endpoints return structured JSON responses used by the frontend application. For example get_reports.php:


```
[
  {
    "id": 1,
    "category_name": "Glass",
    "location": "...",
    "status": "pending"
  }
]
```


HTML response: filter_reports.php: returns HTML fragments for AJAX filtering, the frontend injects the HTML with specific data.

Plain text: throughout the application there are plain text responses: such as “echo “Unauthorized”;” or “echo “Success”;” .

**Security:** The security part of the application ensures that user data and system functionalities are protected against common web vulnerabilities. The application implements multiple practices to ensure security.

- Prepared statements: used for all database interactions. These prevent SQL injection attacks. The SQL queries are separated from user input, making sure that malicious data can't alter the structure of queries.
- User-generated content is sanitized using functions such as htmlspecialchars before displaying in the browser. This ensures that any potential harmful scripts and cross-site scripting XSS attacks are prevented.
- Passwords aren't stored in plain text. They are hashed using password_hash, which applies a hashing algorithm. During login, the passwords are verified using password_verify to ensure secure authentication.
- Authentication mechanisms: users can register and log in, after which a session is created using PHP sessions. It stores essential user data such as user ID and role.
- Authorization: is done based on the user's role (user, worker, admin). Access to certain pages and actions is restricted, each type of user can access designated pages and can perform specific operations depending on the role they have.
- Data validation: found both on the frontend and backend. On the frontend: HTML input fields have constraints (e.g. required fields, valid email formats). On the backend: validation ensures that all received data is correct, and prevents invalid or malicious input.
- Session security: done by regenerating session IDs after login with session_regenerate_id), and reduces the risk of session fixation attacks.

Overall, the application uses the main security practices to protect user data, ensure safe communication and maintain system integrity.

**System Logs & Security Tracking:**

It is important to mention that the System Logs module was built specifically to secure the login and logout information for every user, allowing administrators to know exactly who was active on the platform at any given time.

The core idea is that, in a real production environment, if this site were fully deployed, these logs should ideally be saved in separate daily files that reset or archive at the end of each day. For the current scope of the project, its main purpose is to provide basic accountability and ensure there are no unmonitored situations.

Even though it does not perform overly complex verifications right now, it acts as a crucial information backup. In case something unexpected happens, the system provides a clear trail to see who was online or exactly who performed a specific action within the platform.

**Filters and Search:**

The backend of the application supports filtering and search, allowing retrieving reports or users based on specific criteria.

Filtering is implemented by applying conditions in SQL queries, giving the possibility to view reports that match certain attributes.
Main filtering criteria include:

- Category: Reports can be filtered based on waste type
- Status: Reports can be filtered by their current status
- Location: Reports can be filtered based on their location or address
- Role: Users can be filtered based on their roles

These filters are processed in the backend by dynamically constructing SQL queries using prepared statements. This helps flexibility, and security, as user input is handled safely and SQL injections are prevented.
Search functionality allows users to find specific reports by keywords, location or description. It's implemented using SQL operators such as LIKE and so partial matching of the input text with the text stored.
Filtering and search parameters are received from the frontend via API requests (GET or POST), processed by a controller and passed to the model layer, where the database query is executed. The results are returned in JSON format and displayed dynamically on the frontend, allowing users to interactively explore and manage reports.

## 9. Frontend

The frontend of the application is responsible for user interaction and the visualization of data. It provides an accessible and intuitive interface, allowing efficient interaction between the user and the system. It is developed using standard web technologies: HTML for structure, CSS for styling and JavaScript for interactivity and dynamic behaviour. It communicates with the backend through API endpoints using AJAX (fetch), retrieves data in JSON format and updates the interface without reloading the pages.

**9.1. Pages:** The application contains multiple pages, each designated for specific user roles and functionalities.

**Index:** The index page is represented by the landing page of the application. It shows the map with the current waste report pins, where users can visualize reports and their details. It has a Login Button, in case the user has an account already and a Sign Up button, for the user to register. After scrolling through the sections, the user can find a bigger CTA button, inviting the user to Create an account.

**Register**: this page allows new users to create an account by filling out a form. Input validation is performed both on the frontend (Shows error messages.) and on the backend, to ensure that the data is correct and secured. The default role of a user is "user". The roles can be changed by admins in their dashboards or directly in the database.

**Login**: this page invites the user to authenticate by typing their credentials. Input validation is performed both on the frontend (Shows error messages.) and on the backend, to ensure that the data is correct and secured. After a successful login, a session is created and users are redirected to their dashboards, based on their roles.

**Used dashboard**: Allows a regular user to view all reports on map and track the status of their own reports. The main button on the user dashboard is the “Report Waste” button which leads to a form page. The form has the following inputs: a select option for the category of the waste, a text input for description of the issue, and an autofill field which is completed by selecting a location on the map.

**Worker dashboard:** This page is specific for the authorized personnel. These users can view reports, can accept tasks, update report statuses and mark issues as resolved, and archive them.

**Admin dashboard**: This page provides admin specific functionalities such as managing users, viewing all reports, accessing statistics, generating statistical and analytic reports, editing all types of users and controlling all types of reports.

**9.2. Styling**: This web application uses CSS for styling and layout. General rules are applied throughout all pages, and specific rules for specific pages. Reusable classes and components are used to ensure visual consistency of overall application.

**9.3. JavaScript**: is used for interactivity and dynamic behaviour. It’s responsible for handling user events, updating the DOM and communicating with the backend via AJAX requests.
Functionalities:

- Fetching and displaying data from API endpoints
- Handling user interactions (e.g. click events, form submissions)
- Updating UI elements dynamically without page reload
- Managing the interactive map, Leaflet integration

**9.4. Responsiveness**:
The application is designed to be responsive and accessible across different devices and screen sizes. Responsive design techniques: flexible layouts, media quories, adaptive components, ensure correct usability on desktop, tablets or mobile devices. A responsive navigation menu (hamburger menu) is implemented for smaller screens, improving user experience.

## 10. Asynchronous JavaScript and XML (AJAX)

To ensure asynchronous communication between the frontend and the backend without requiring a page reload, the application uses AJAX.
AJAX is implemented using fetch API in JavaScript: allows the application to send HTTP requests to backend API endpoints and receive data in JSON format.
AJAX is used for functionalities as:

- Retrieving reports from the backend and displaying them dynamically on the map
- Filtering and searching reports and users without reloading the page
- Updating report statuses (marking a report as ‘in progress’, ‘pending’ or ‘resolved’)
- Performs user and report actions without reloading the page

- Loads data dynamically in dashboards.

AJAX improves performance and user experience, avoids unnecessary page reloads and enables dynamic updates of the interface.
For example, when loading reports on the map, the frontend sends a GET request to the API endpoint, receives a JSON response and dynamically creates markers using JavaScript.
Another example, when a user or worker performs an action (e.g. resolving a report), a POST request is sent to the backend, which processes the request and returns a response, allowing the interface to update instantly.

### 11. Map

The map component is an essential feature of this application: it allows users to view waste reports from a geographical point of view. It provides an interactive interface where users can see locations with accumulated garbage, see the report details, what user reported it, what is the status of it.
The map is implemented using the Leaflet.js open-source JavaScript library for interactive maps. The map is centered on a predefined location, and it displays pin markers of reported waste areas. Each pin has coordinates: latitude and longitude, which are stored in the database. Each waste category has a different color pin. Clicking a pin opens an overlay displaying the detailed information about the report: this interaction is handled using JavaScript event listeners and DOM manipulation.
The data is dynamically loaded from the backend, using AJAX requests: fetch API, retrieving report data in JSON format from the API endpoints. This ensures that the map is updated without loading the page and provides responsive user experience.
The map has the “Locate Me” functionality: uses the browser’s Geolocation API to detect the user’s current location, making it easier and effortless to report nearby waste issues.
Overall, the map is a central feature of the application, provides clear and interactive visualization of waste reports, and benefits both citizens and authorized personnel in managing problematic areas more efficiently.

### 12. Status flow

The application manages a status flow of the waste reports. Each report goes through a series of states in the solving process.
Possible statuses of reports are:

- Pending: The report has been submitted by a user and is waiting to be reviewed or assigned to a worker. This status is automatically assigned to a newly submitted report.
- Progress: The report is currently handled by a worker: a clean-up team has been sent to the area.
- Resolved: The issue has been successfully cleared.
- Archived: The report has been archived by the authorized personnel and is marked as completed.

The status of each report is stored in the database and can be seen in the user interface. Users can track the status of their own reports and workers and administrators can update the status through the action buttons of their dashboards.

### 13. Reports and Statistics:

The application includes a reporting and analytics section designed to provide insights regarding waste accumulation, report activity and environmental cleanliness across different areas.
Reports can be **generated automatically or manually** for different time intervals (Daily/Weekly/Monthly Reports).
**Generated reports contain** relevant information such as: Report location, Waste category, Description of the issue, Current report status, Reporting user, Creation dates.
**Export**: The application supports data export functionalities using standard format such as CSV, PDF or JSON. JSON is used for the frontend - backend communication. The API endpoints return data in JSON format, which is easily received by JavaScript. This allows dynamic loading of data without reloading the page, improves performance and user experience. CSV is used for exporting report data in a format that can be opened in spreadsheet readers such as Microsoft Excel or Google Sheets. The exported files contain report information, such as report ID, category, location, status or creation date. PDF is used for printable and shareable exports.
Visualisations: the application provides statistical dashboards and graphical visualizations used for monitoring waste management activity and identifying problematic areas. These display: total number of reports, reports grouped by category, report status distribution, evolution of reports over time, cleanest and dirtiest areas, worker activity and report resolution progress.
The frontend uses SVG, CSS and JavaScript for rendering interactive charts and graphs, including: bar charts, donut charts, line charts, dynamic statistical graphics.

### 14. Testing

The first category of testing the application is testing **from the user interface** perspective to ensure correct functionality, usability and responsiveness across different scenarios.
Testing focused on verifying that all pages and interactive components behave as expected when used by different types of users (regular users, authorized personnel and administrators).
The following UI components and functionalities were tested:

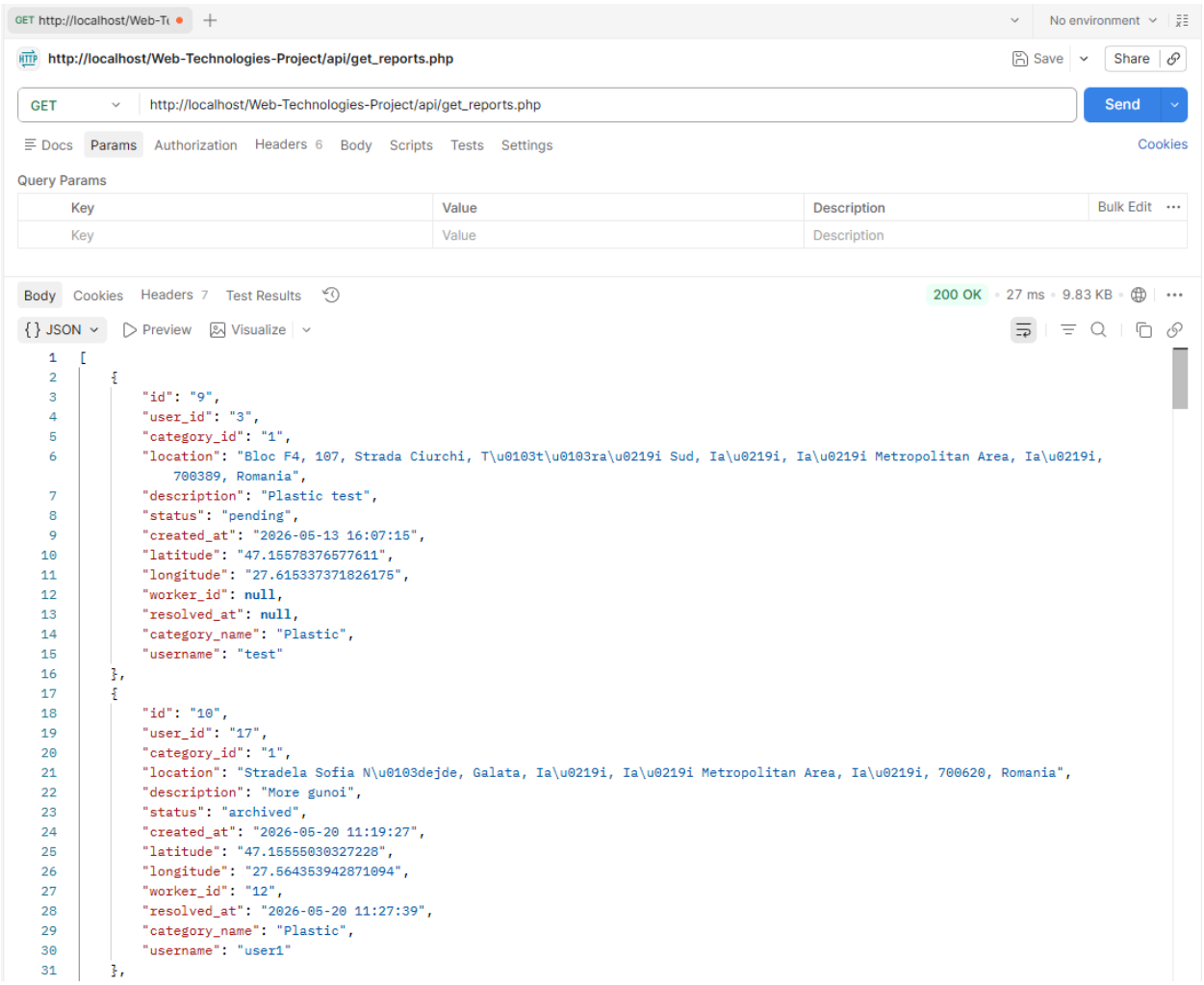
- Navigation: All pages were tested to ensure correct routing and accessibility.
- Forms: Input forms (register, login, report submission) were tested for validation, error handling and correct data submission.
- Authentication: Login and logout were tested to verify session creation, role-based redirection and access restrictions.
- Dashboards: Each dashboard was tested to confirm that the correct data is displayed, clickable and shows accurate report information.
- Status updates: Changing report statuses was tested to verify correct UI updates and backend communication.
- Filtering and search: Filtering reports by category, status and location was tested to ensure accurate results.
- Responsive design: The application was tested on different screen sizes to ensure proper layout adaptation.
- User interactions: button clicks, form submissions, dynamic updates were verified using JavaScript event handling and AJAX requests.

- Error and edge cases: empty inputs, invalid data, unauthorized access were considered to ensure that the application handles each case scenario correctly.

The second type of testing the application is using the **Postman** application. To verify the functionality and correctness of the backend API, the application was tested using Postman. Postman was used to simulate HTTP requests between the frontend and backend components and to validate the behaviour of the API endpoints under different scenarios.

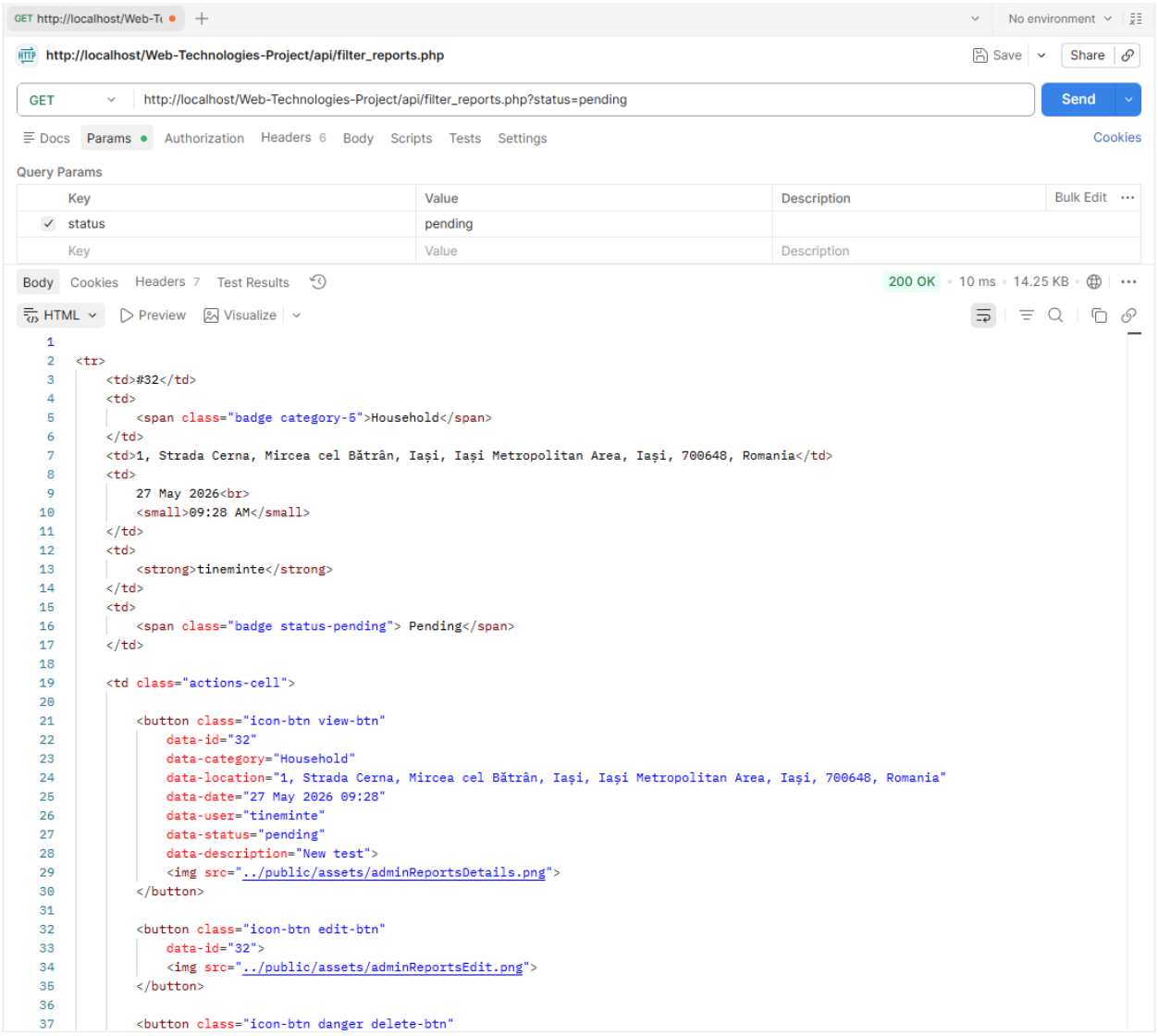
Examples of API testing with Postman:

1. *http://localhost/Web-Technologies-Project/api/get_reports.php*



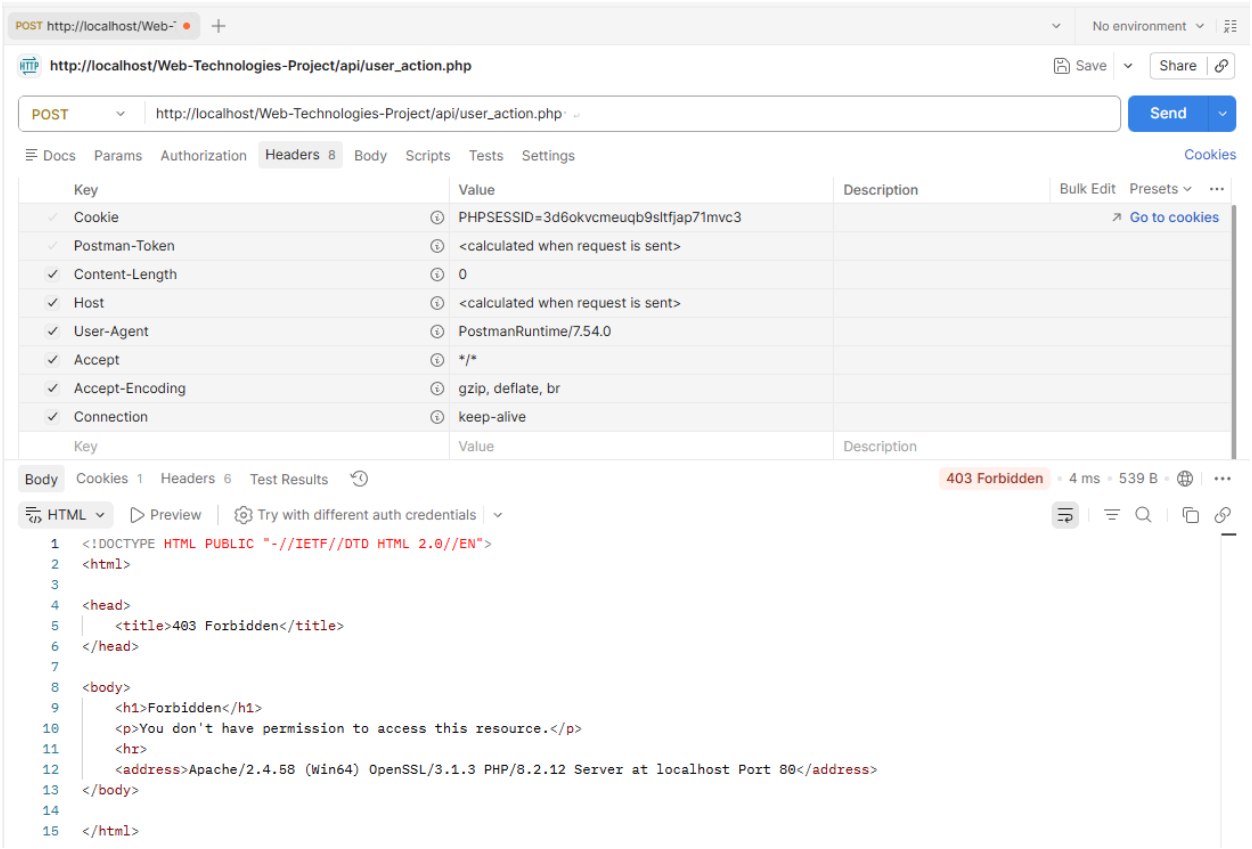

GET request for retrieving reports from the backend API.
The endpoint returns report data in JSON format, including category, location, status, coordinates and assigned worker.
The response status 200 OK confirms successful communication between frontend and backend.

2. *http://localhost/Web-Technologies-Project/api/filter_reports.php?status=pending*



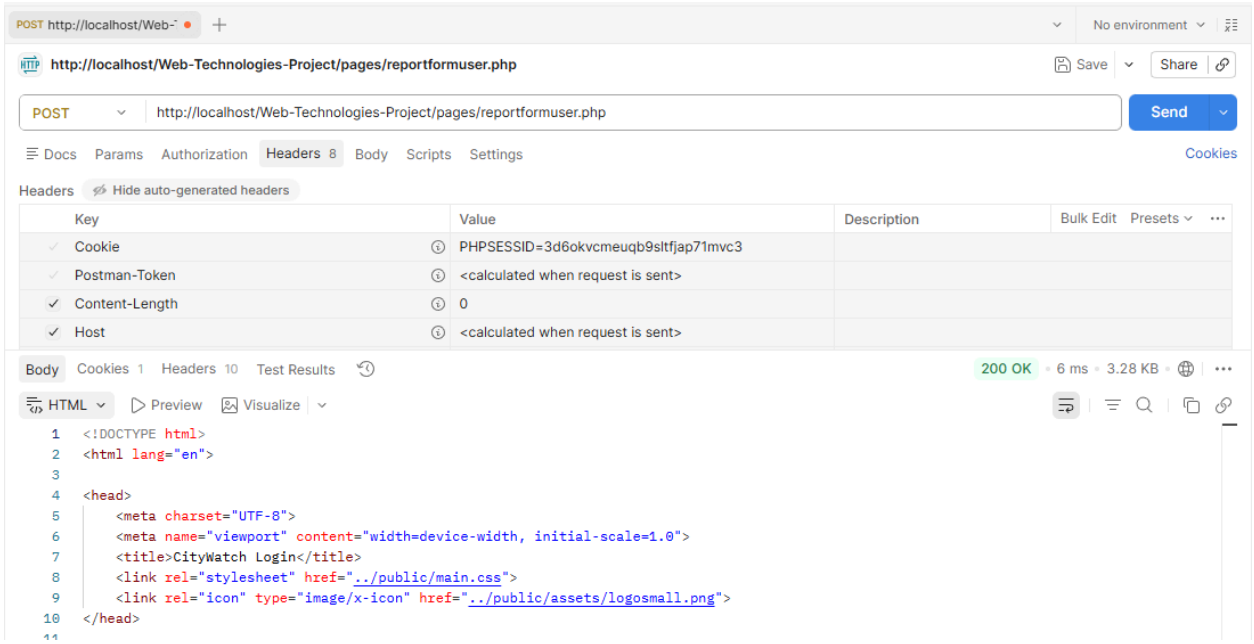

GET request used for filtering reports based on their current status.
The endpoint receives query parameters and dynamically filters report data from the database using prepared SQL statements.
The response contains only reports marked with the “pending” status, demonstrating asynchronous filtering functionality implemented with AJAX and API communication.

3. *http://localhost/Web-Technologies-Project/api/user_action.php*



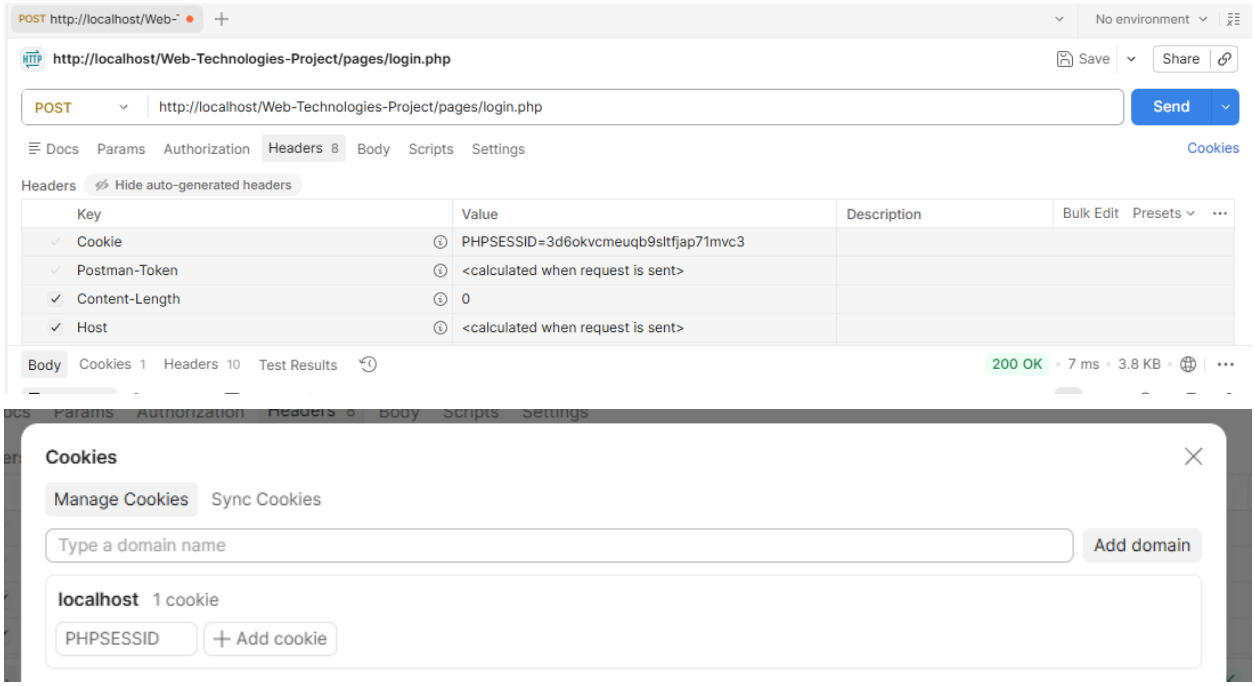

POST endpoint protected through authentication and role-base authorization.
Unauthorized requests without a valid session are rejected with HTTP 403 Forbidden response. This ensures that only administrators can perform sensitive actions such as deleting or modifying users.

4. *http://localhost/Web-Technologies-Project/pages/reportformuser.php*




POST request used for submitting a new waste report through the user reporting form. The page allows users to provide information such as waste category, location, description and coordinates selected on the interactive map. Submitted data is validated and sent asynchronously to the backend, where it's processed and stored in the database.

5. *http://localhost/Web-Technologies-Project/pages/login.php*




Cookie storage: The application creates and stores both the PHP session cookie (PHPSESSID) and a custom persistent authentication cookie (remember_user) used for the "Remember Me"

functionality. These cookies allow secure session management and persisting user authentication across multiple visits.

**15. Conclusions**

The CityWatch application successfully demonstrates the development of a web platform for waste management and community reporting. The project integrates frontend and backend technologies: HTML, CSS, JavaScript, PHP and MySQL to provide an interactive and responsive user experience. It integrates Leaflet.js to provide interactive geographic maps.

This application supports multiple user roles: users, workers and admins, each having different functionalities and access permissions.
Users can submit waste reports, visualize issues on an interactive map, track report statuses and interact with the system. Workers can manage interventions and update report statuses, while administrators can manage reports, users and system statistics.

The project implements web development concepts such as session management, cookies, AJAX communication, REST API endpoints, asynchronous data loading, role-base access control and security mechanisms against SQL injection and XSS attacks.